ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL — ESPOL

Organización de Computadores

PROYECTO ASSEMBLY

Conversor de Bases Numéricas

Emulador: emu8086 — Assembly x86 de 16 bits

Integrantes: Fátima Gabriela Torres Vera

César Andres Balladares Caicedo

PAO 1 2026

8 de julio de 2026

1. Descripción General

Este proyecto implementa, en lenguaje Assembly de 16 bits para el emulador **emu8086**, un programa interactivo que permite convertir un número entero positivo entre distintas bases numéricas. El programa presenta un menú con cuatro opciones de conversión y una opción de salida, validando en todo momento el ingreso del usuario y permitiendo realizar múltiples conversiones sin necesidad de reiniciar la ejecución.

Las conversiones soportadas son:

- **Decimal → Binario** : lee un número decimal (0–255) y muestra su representación binaria de 8 bits.
- **Decimal → Hexadecimal**: lee un número decimal (0–255) y muestra su representación hexadecimal (2 dígitos).
- **Binario → Decimal**: lee una cadena de `0`s y `1`s (hasta 8 dígitos) y calcula su valor decimal equivalente.
- **Hexadecimal → Decimal**: lee una cadena hexadecimal (dígitos `0-9` y letras `A-F`/`a-f`, hasta 2 dígitos) y calcula su valor decimal equivalente.

Toda la entrada del usuario se lee como cadena de caracteres mediante `INT 21h`, función `0Ah` (entrada bufferizada), y se valida **carácter por carácter** antes de procesarla, según la base de entrada correspondiente. Si se detecta un carácter inválido, una longitud incorrecta, o — en el caso de la entrada decimal — un valor fuera del rango permitido (0 a 255), el programa muestra un mensaje de error claro y vuelve a solicitar el dato, **sin interrumpir la ejecución en ningún momento**.

Cada resultado se imprime junto con una etiqueta que indica explícitamente la base de salida (por ejemplo, *Resultado en BINARIO:"*), y al finalizar cualquier conversión el programa regresa automáticamente al menú principal.

2. Instrucciones de Uso

1. Abrir el emulador **emu8086**.
2. Cargar el archivo `conversor.asm`.
3. Compilar el programa con el botón *Compile*.
4. Ejecutarlo en el emulador (botón *Emulate* o *Run* , según la versión.)
5. El programa mostrará un menú principal interactivo con las 5 opciones disponibles.
6. Digitar el número correspondiente a la opción deseada (1–5) y presionar `Enter`.
7. Ingresar el valor solicitado según la base indicada:
   - *Decimal*: solo dígitos `0-9`, entre 0 y 255 (máximo 3 dígitos).
   - *Binario*: solo caracteres `0` y `1`, hasta 8 bits.
   - *Hexadecimal*: dígitos `0-9` y letras `A-F` (mayúscula o minúscula), hasta 2 dígitos.

8. Si el dato ingresado no es válido (carácter fuera de la base, longitud excesiva o valor fuera de rango), el programa muestra un mensaje de error y solicita el dato nuevamente, sin terminar la ejecución.

9. El resultado se imprime en pantalla junto con una etiqueta que indica la base de salida (BINARIO, HEXADECIMAL o DECIMAL).

10. Tras mostrar el resultado, el programa regresa automáticamente al menú principal.

11. Seleccionar la opción `5` para finalizar el programa.

3. Estructura del Código

El programa se organiza en procedimientos independientes (`PROC`/`ENDP`), cada uno con un encabezado que documenta su objetivo y los registros que modifica. La siguiente tabla resume su función y la instrucción clave que utiliza:

Procedimiento	**Instrucción clave**	**Función**
`MENU_PRINCIPAL`	INT 21h (01h)	Muestra el menú y lee la opción elegida por el usuario.
`LEER_NUMERO`	INT 21h (0Ah), CMP	Lee la cadena de entrada y valida carácter a carácter según la base activa.
`DEC_A_BIN`	SHR, AND	Convierte un valor de 0-255 a su cadena binaria de 8 bits.
`DEC_A_HEX`	SHR, AND	Convierte un valor de 0-255 a su cadena hexadecimal de 2 dígitos.
`BIN_A_DEC`	DIV	Convierte el valor ya interpretado desde binario a su cadena decimal.
`HEX_A_DEC`	DIV	Convierte el valor ya interpretado desde hexadecimal a su cadena decimal.
`NUM_A_CADENA_DEC`	DIV, PUSH/POP	Rutina auxiliar: extrae dígitos decimales por resto y usa la pila para invertir su orden.
`MOSTRAR_RESULTADO`	INT 21h (09h)	Imprime la etiqueta de salida junto con el resultado.
`MOSTRAR_ERROR`	INT 21h (09h)	Imprime el mensaje de entrada inválida antes de reintentar.

Durante la validación de entrada, la conversión de un carácter ASCII a su valor numérico sigue la siguiente equivalencia:

Carácter	**Código ASCII**	**Valor numérico**	**Operación**
`'0'--'9'`	30h–39h	0–9	`AL - 30h`
`'A'--'F'`	41h–46h	10–15	`AL - 37h`
`'a'--'f'`	61h–66h	10–15	`AL - 57h`

4. Manejo de Errores

El programa nunca finaliza abruptamente ante una entrada incorrecta. En `LEER_NUMERO`, cada carácter se compara contra el rango válido de la base activa (dígitos para decimal, `0/1` para binario, `0-9/A-F/a-f` para hexadecimal); también se valida la longitud máxima de la cadena y, en el caso decimal, que el valor resultante no supere 255. Si cualquiera de estas comprobaciones falla, se llama a `MOSTRAR_ERROR`, se imprime el mensaje correspondiente y el flujo regresa a `REINTENTAR_LECTURA` para solicitar el dato de nuevo, cumpliendo el requisito de que *el programa no debe fallar ante ingresos incorrectos del usuario.*

5. Capturas de Pantalla de Ejecución

Menú Principal

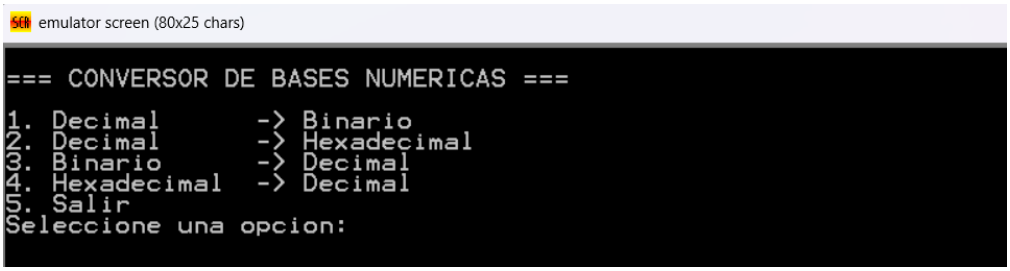


Figura 1: Pantalla inicial del menú principal.

1. Decimal a Binario

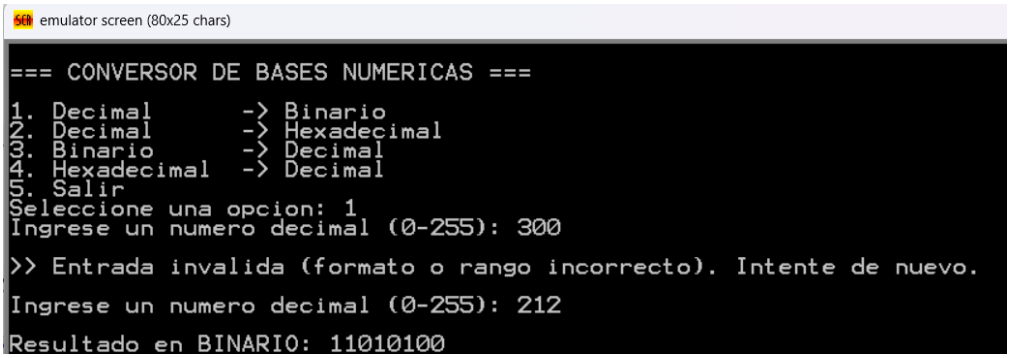


Figura 2: Ejecución de la opción 1: Decimal a Binario.

2. Decimal a Hexadecimal

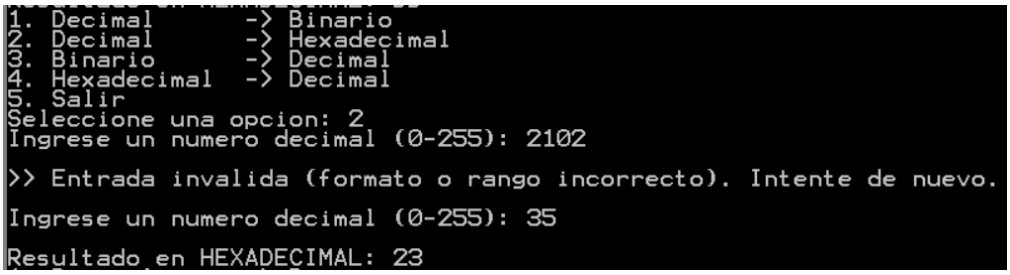


Figura 3: Ejecución de la opción 2: Decimal a Hexadecimal.

3. Binario a Decimal

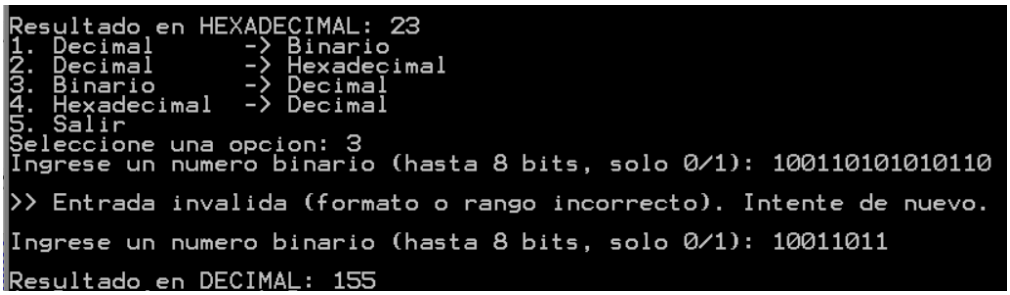


Figura 4: Ejecución de la opción 3: Binario a Decimal.

4. Hexadecimal a Decimal

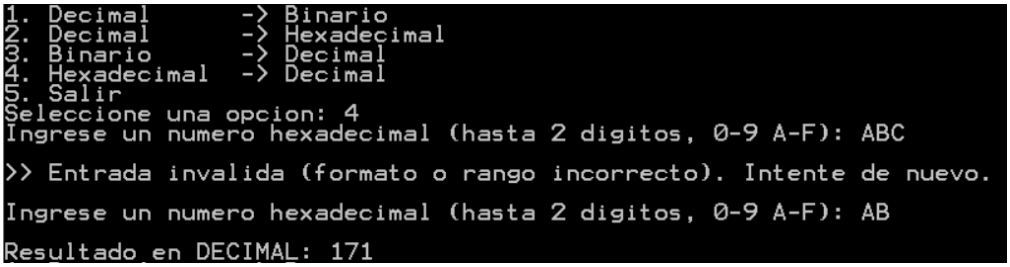


Figura 5: Ejecución de la opción 4: Hexadecimal a Decimal.

Manejo de Entrada Inválida

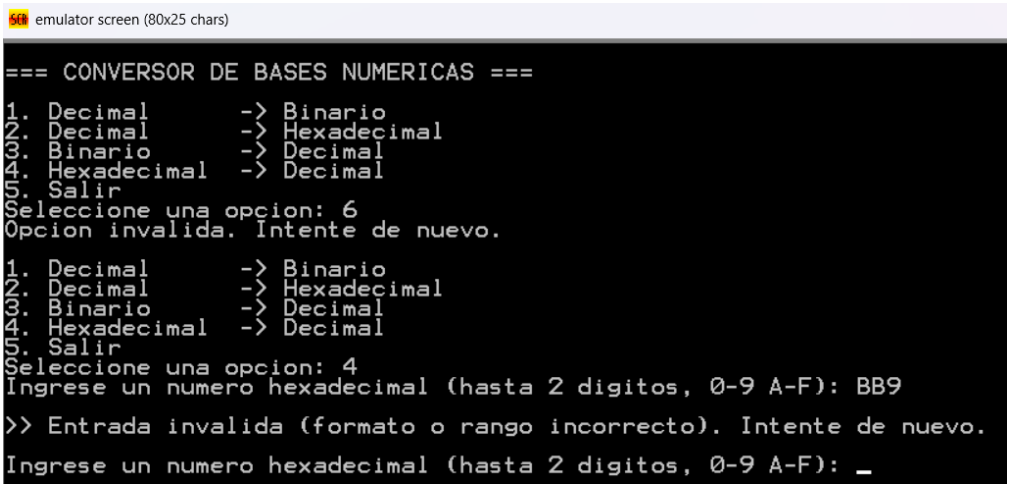


Figura 6: Validación de un ingreso incorrecto y reintento de lectura.

5. Salir

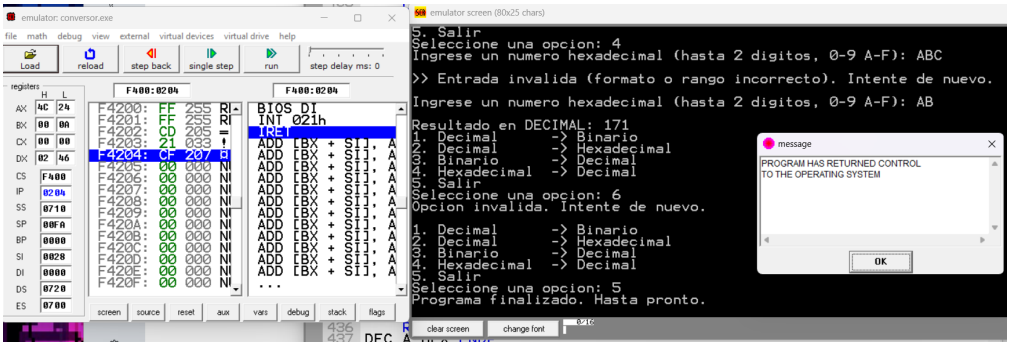


Figura 7: Ejecución de la opción 5: Salir del programa.

6. Conclusiones

El desarrollo de este proyecto permitió aplicar de forma práctica los conceptos fundamentales de la programación en Assembly x86: manejo directo de registros, direccionamiento de memoria, y las instrucciones aritméticas y lógicas de bajo nivel (DIV, MUL, SHL, SHR, PUSH/POP) necesarias para convertir valores entre distintas bases numéricas. Modularizar la solución en procedimientos independientes, cada uno con una responsabilidad clara, facilitó la comprensión del flujo completo del programa. Asimismo, implementar una validación robusta de la entrada del usuario reforzó la importancia de nunca asumir que los datos ingresados son correctos, un principio esencial en el diseño de cualquier programa confiable, incluso en el nivel más bajo de abstracción como es el lenguaje ensamblador.